Rollspot V1.1

Software Engineering Documentation & System Architecture Audit

SOURCE: DOCS/README.MD

Rollspot engineering documentation

This is the code documentation of the V1.1.

Status labels

- **Verified** — confirmed in source, configuration, tests, or a command executed during the audit.
- **Inferred** — strongly suggested by the implementation, but not explicit.
- **Unknown** — not recoverable from the repository.
- **Potential issue** — an evidenced inconsistency or risk; not necessarily a confirmed defect.

Start here

Area	Document
System architecture	`architecture/system-overview.md`
Components	`architecture/components.md`
End-to-end flows	`architecture/data-flow.md`
Integrations	`architecture/integrations.md`
Deployment topology	`architecture/deployment.md`
Development setup	`getting-started.md`
iOS	`ios/README.md`
Android and parity	`android/README.md`
Backend internals	`backend/README.md`
Relational model	`backend/data-models.md`
API inventory	`api/README.md`
Security and privacy	`security.md`
Testing and quality	`testing.md`
Deployment and runbooks	`operations/README.md`
Architecture decisions	`engineering/architecture-decisions.md`
Technical debt	`engineering/technical-debt.md`
Partner maintenance summary	`operations-and-failure-modes.md`

Verified inventory

Component	Location	Technology	Responsibility	Direct dependencies
Backend API	`backend/`	Cloudflare Worker, TypeScript 5.9, Better Auth 1.7	HTTP API, auth, validation, grade aggregation, scheduled place sync	D1, R2, Resend, Apple, Google, Overpass

Component	Location	Technology	Responsibility	Direct dependencies
Shared mobile logic	`Mobile/shared/`	Kotlin 2.4, KMP, Ktor 3.6, coroutines/Flow, SKIE	API client, DTOs, state models, validation, review orchestration	Backend API
iOS app	`Mobile/Rollspot/`	SwiftUI, MapKit, Shared static framework	Native UI, Apple map rendering, permissions, native identity and media pickers	Shared, Apple frameworks, Google Sign-In
Android app	`Mobile/android/`	Kotlin, Compose, MapLibre 13.6	Native UI, OSM map rendering, permissions, Credential Manager	Shared, OpenFreeMap, Google identity
Website	`web/`	Static HTML/CSS/JS, AASA	Landing and place links	Cloudflare Pages (verified from live response; deployment source unknown)
Database	`backend/migrations/`	Cloudflare D1 / SQLite	Auth, places, reviews, signals, profiles, saved places, sync state	Worker
Media	R2 binding in `wrangler.jsonc`	Cloudflare R2	Review photos and avatars	Worker

Boundaries

Business rules and all backend calls are intended to live in `Mobile/shared`; iOS and Android remain native presentation layers. Verified exceptions and legacy code are recorded in `engineering/technical-debt.md`.

No WebSocket, GraphQL, gRPC, message queue, push-notification service, offline database, or mobile background worker is implemented in the inspected repository.

Getting started

Prerequisites

Tool	Verified requirement
Node/npm	Required; no version is pinned
JDK	JDK 25 path is used by repository instructions; Gradle compiles Android to JVM 17
Gradle	Wrapper 9.1.0 in `Mobile/gradle/wrapper`
Xcode	iOS 18 SDK capable; exact Xcode version is not pinned
Android SDK	compile/target SDK 36, min SDK 26
Wrangler	locked to 4.131.0 by `backend/package-lock.json`

Backend

```
cd backend
npm install
cp .dev.vars.example .dev.vars
npm run db:migrate:local
npm run dev
```

Local API: `http://localhost:8787`. Auth emails are logged by the local Worker when `RESEND_API_KEY` is absent. Do not commit `.dev.vars`.

Useful checks:

```
npm run check                   # TypeScript compilation
npm test                        # currently the same compilation check
npm run db:migrate:remote       # production migration; requires Cloudflare auth
npm run deploy                  # Worker deployment; requires Cloudflare auth
```

Shared module and Android

```
export JAVA_HOME=/opt/homebrew/opt/openjdk@25/libexec/openjdk.jdk/Contents/Home
cd Mobile
./gradlew :shared:testDebugUnitTest :shared:iosSimulatorArm64Test
./gradlew :androidApp:assembleDebug
```

For the Android emulator, place these uncommitted values in `Mobile/local.properties`:

```
rollspot.apiBaseUrl=http://10.0.2.2:8787
rollspot.googleWebClientId=YOUR_PUBLIC_WEB_CLIENT_ID
```

Use `Mobile/scripts/run-android-debug.sh` to start the configured emulator workflow before installing a debug build. The `Mobile/android/EMULATOR.md` path mentioned in older repository guidance is not present at this baseline.

iOS

Open `Mobile/Rollspot.xcodeproj`. The first Xcode build phase invokes `Mobile/scripts/build-shared-for-xcode.sh`, which builds and embeds the static `Shared` KMP framework.

```
xcodebuild -project Mobile/Rollspot.xcodeproj -scheme Rollspot \
  -destination 'generic/platform=iOS Simulator' ARCHS=arm64 build
```

For a local backend, set the Run scheme environment variable:

```
ROLLSPOT_API_BASE_URL=http://localhost:8787
```

Google Sign-In also requires replacing the public placeholders in `Mobile/Info.plist`. Sign in with Apple must be enabled for `com.puspadifellas.app`.

Live integration test

Against a disposable/local backend:

```
cd Mobile
ROLLSPOT_LIVE_URL=http://localhost:8787 \
  ./gradlew :shared:testDebugUnitTest --tests '*LiveBackendTest*' --rerun
```

The suite creates and deletes account/review data. Do not target production without explicit approval.

Environments

Environment	API selection
Worker local	Wrangler on port 8787
iOS debug	`ROLLSPOT_API_BASE_URL`, otherwise production default
Android local	`rollspot.apiBaseUrl` in `Mobile/local.properties`
Production default	`https://rollspot-api.joecast208.workers.dev`

There is no tracked staging environment or CI workflow. Production account access, secret presence, and deployed migration state are **Unknown** from the repository.

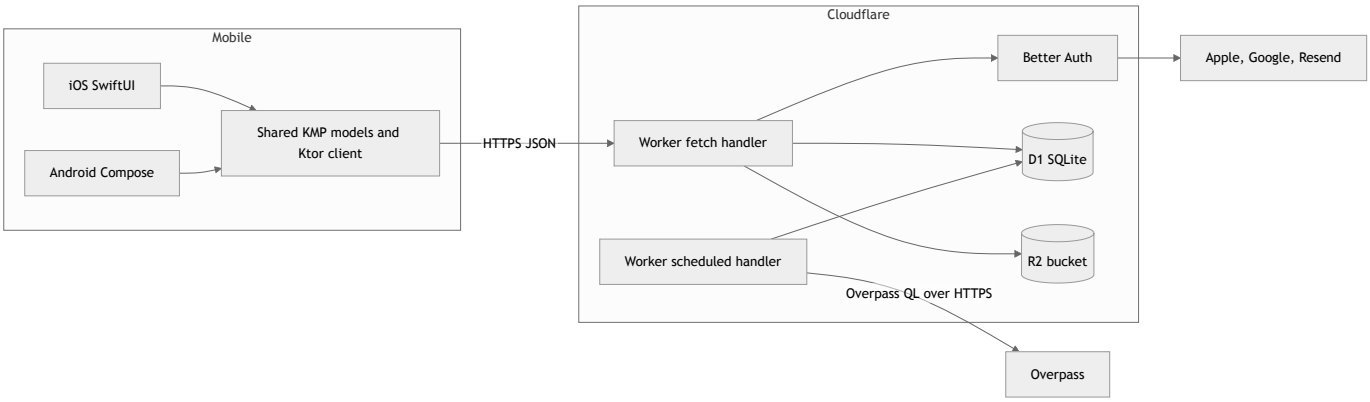
SOURCE: DOCS/ARCHITECTURE/SYSTEM-OVERVIEW.MD

System overview

Purpose

Rollspot lets mobility-impaired users discover places and publish structured first-hand observations about entrances, elevators, toilets, pathways, seating, and parking. V1.1 is one serverless backend, one shared mobile business-logic module, and two native mobile interfaces.

Containers



No queue or cache layer exists. JSON responses use `Cache-Control: no-store`; media uses `no-cache`. The apps do not maintain an offline database and do not receive server-pushed updates.

Architectural responsibilities

Component	Responsibility	State owned	Public interface	Main failure effect
iOS	SwiftUI screens, MapKit rendering, location, camera/photos, native sign-in	Keychain token, draft in UserDefaults, image cache	Shared models and deep links	One-platform UI or native capability failure
Android	Compose screens, MapLibre rendering, location, photo picker, native Google sign-in	Encrypted token, one preference flag, in-memory drafts	Shared models and custom links	One-platform UI, tile, or lifecycle failure
Shared KMP	Validation, state machines, API/DTO boundary, retries, grade rules, submission orchestration	In-memory `StateFlow` state	`RollspotSdk` and feature models	Usually affects both apps
Worker	Routing, auth, authorization, validation, persistence, public reads, cron	None outside bindings	HTTP `/api/auth/*`, `/v1/*`, `/health`	Both apps lose affected server feature
D1	Relational source of truth	Users, places, reviews, signals, profiles, sync metadata	Worker SQL only	API data failure
R2	Binary object source of truth	JPEG avatars and review photos	Worker binding only	Photo upload/read failure
Website	Link landing and iOS association file	Static assets	HTTPS pages and AASA	Shared links fall back or fail

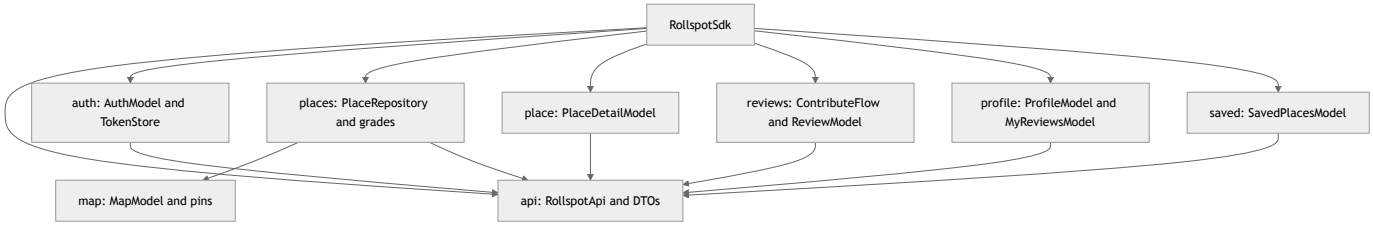
Verified patterns

- Manual dependency injection: each app creates one `RollspotSdk`.
- Unidirectional state: shared models expose `StateFlow`; native views send actions.
- Repository/model boundary: `PlaceRepository`, `MapModel`, `PlaceDetailModel`, and other feature models sit between UI adapters and `RollspotApi`.
- Idempotent writes: stable review submission IDs, stable media keys, and idempotent saved-place PUT/DELETE.
- Native views/shared logic: platform SDK and presentation remain native; portable decisions belong in KMP.
- Serverless edge runtime: Worker `fetch` and `scheduled` handlers use D1 and R2 bindings.

SOURCE: DOCS/ARCHITECTURE/COMPONENTS.MD

Component architecture

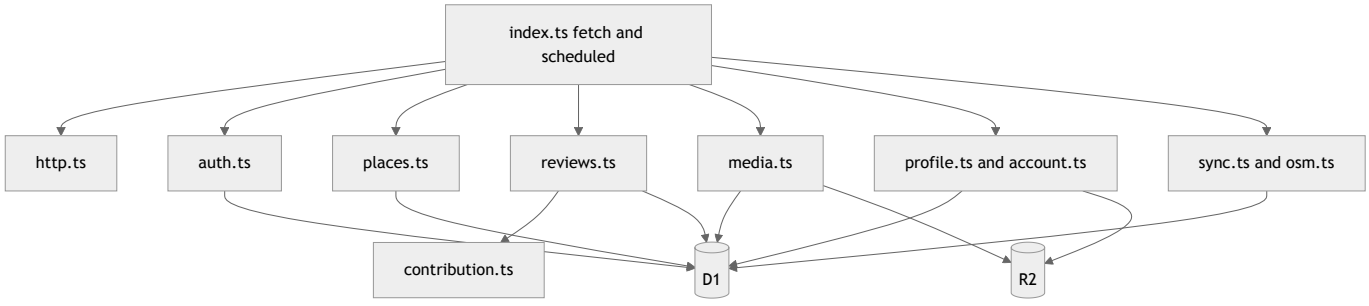
Shared mobile component



`RollspotSdk` in `Mobile/shared/.../RollspotSdk.kt` is the composition root. Models expose `StateFlow` except `ContributeFlow`, whose mutable state is republished by native adapters. `RollspotApi` uses Ktor Darwin on iOS and CIO on Android.

Package	Inputs	Outputs	Failure behavior
`api`	DTOs, token, HTTP request	Decoded DTO or `ApiException`	GET network failures retry at 400/800 ms; writes do not auto-retry
`auth`	credentials/ID tokens, native callbacks	`AuthSession`	Validation is local; 401/session absence signs out
`places`	location/search/place key	place domain models	OSM attribution is carried with results
`map`	viewport, filter, search actions	`MapState`	Error state supports UI retry
`place`	Rollspot place key	detail/reviews/photos state	Concurrent sub-loads degrade independently
`reviews`	answers and JPEG bytes	`ReviewState` /submitted review	Stable IDs make explicit retries safe
`profile` / `saved`	user actions	user-specific state	Optimistic mutations roll back on failure

Backend component



The Worker uses ordered pathname checks rather than a router framework. `fetch` catches `HttpError`, logs unexpected 5xx failures, and emits JSON. `scheduled` invokes `syncDueTiles` with `ctx.waitUntil`.

iOS component

`RollspotApp` creates app-wide `LanguageManager` and `AuthSessionStore`; `ContentView` presents `HomeMapView`. `RollspotServices.sdk` constructs the shared SDK with `CloudflareConfig` and `KeychainTokenStore`.

Adapter	Shared dependency	Native responsibility
`MapStore`, `NearbyPlacesService`	`MapModel`, `PlaceRepository`	MapKit annotations/camera
`AuthSessionStore`	`AuthModel`	Apple/Google SDK UI and callback handling
`PlaceReviewStore`	`PlaceDetailModel`	SwiftUI detail presentation
`ReviewService`, bridges	`ReviewModel`, `ContributeFlow`	Swift data/image conversion
`ProfileService`, `SavedPlacesService`	profile/saved models	SwiftUI observable state

State uses `ObservableObject`, `@Published`, `@StateObject`, and SKIE-generated `AsyncSequence`.

Android component

`RollspotApplication` constructs the shared SDK and restores the session. `MainActivity` hosts `RollspotApp`, whose `NavHost` defines home, detail, saved, auth, profile/settings, and contribution routes.

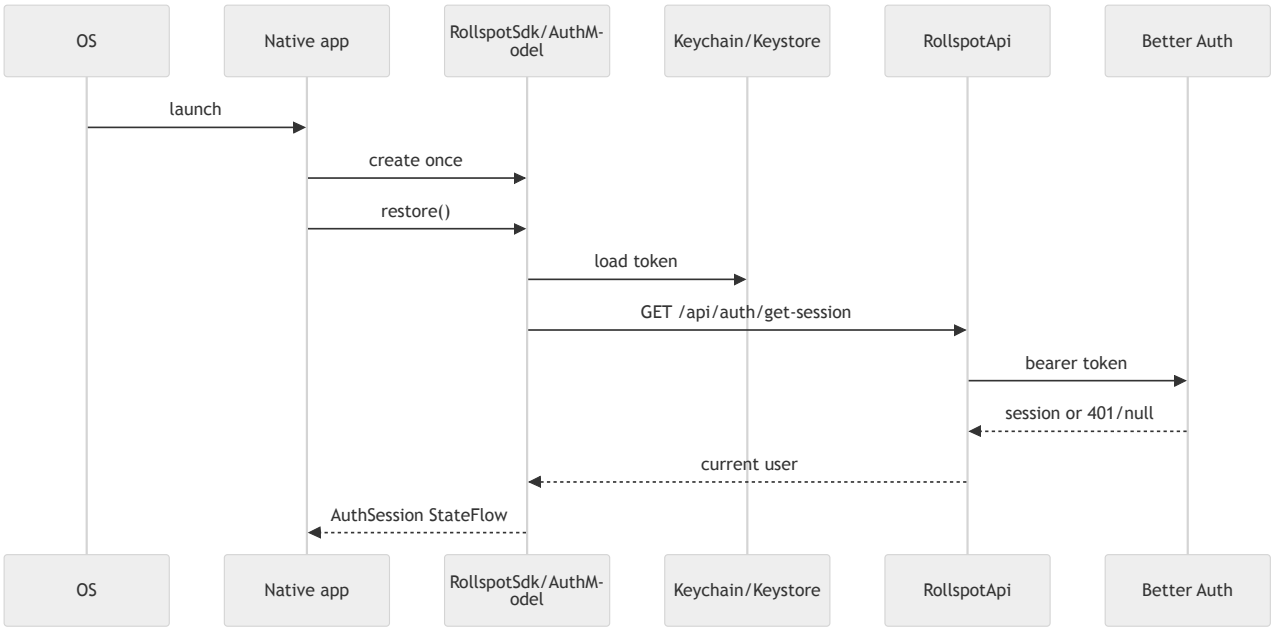
Android adapter	Shared dependency	Native responsibility
`HomeViewModel`	`MapModel`	Compose state and MapLibre events
`PlaceDetailViewModel`	`PlaceDetailModel`	lifecycle-bound detail state
`ContributeFlowViewModel`	`ContributeFlow`, `ReviewModel`	picker bytes and wizard UI
`AuthViewModel`	`AuthModel`	Credential Manager result

Most shared flows use `collectAsStateWithLifecycle`. No DI framework, Room, WorkManager, service, receiver, or notification component is present.

SOURCE: DOCS/ARCHITECTURE/DATA-FLOW.MD

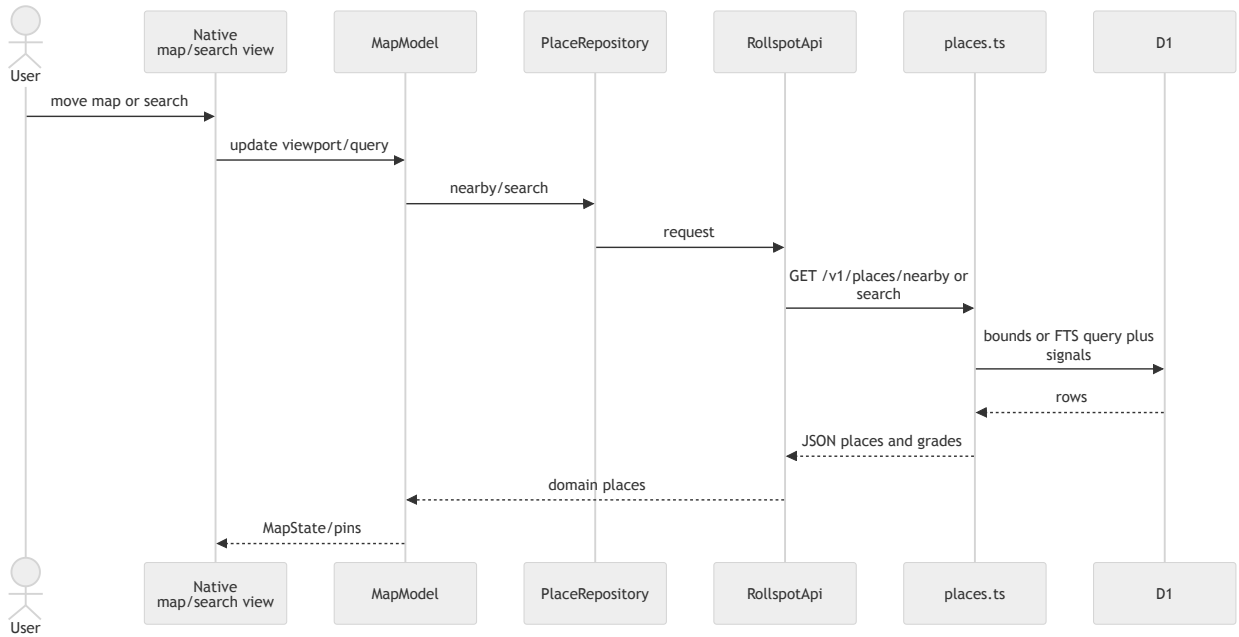
End-to-end data flows

Startup and session restoration



iOS starts with `SplashScreenView`; Android restores in `RollspotApplication.appScope`. User-specific saved, profile, and review state is cleared when the shared session becomes signed out.

Place discovery and detail



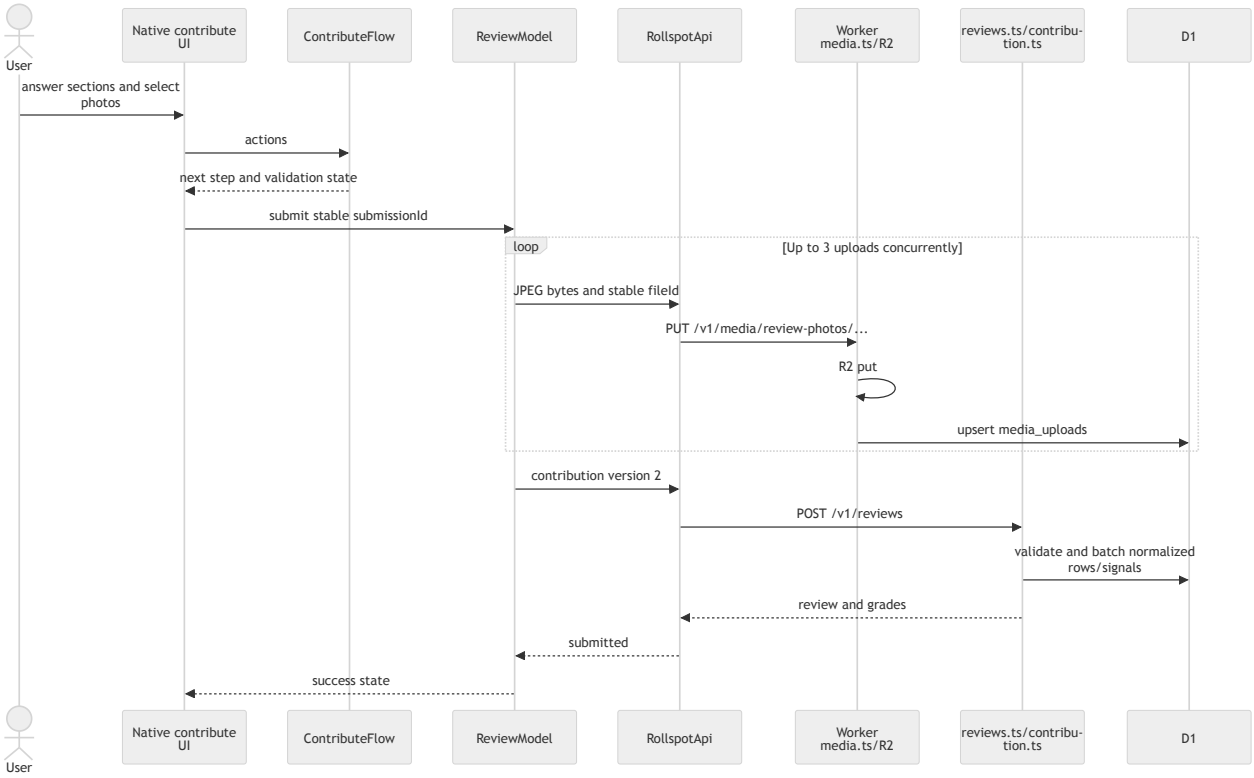
Selecting a Rollspot place creates `PlaceDetailModel`, which concurrently requests the place, venue photos, review photos, and reviews. MapKit/OpenFreeMap provide map visuals, not the place directory.

Authentication

Native Apple/Google controls obtain an identity token; `AuthModel` sends it to `POST /api/auth/sign-in/social`. Email flows use Better Auth email routes and Resend verification. The returned bearer token is stored in iOS Keychain or encrypted with Android Keystore. Authenticated requests attach `Authorization: Bearer`.

Recovery: client validation prevents malformed forms; provider/server failures become `ApiException`. Sign-out clears local state even if the remote sign-out fails. A potential stale-token path remains when `get-session` returns 401/null without `RollspotApi` clearing storage.

Contribution and persistence

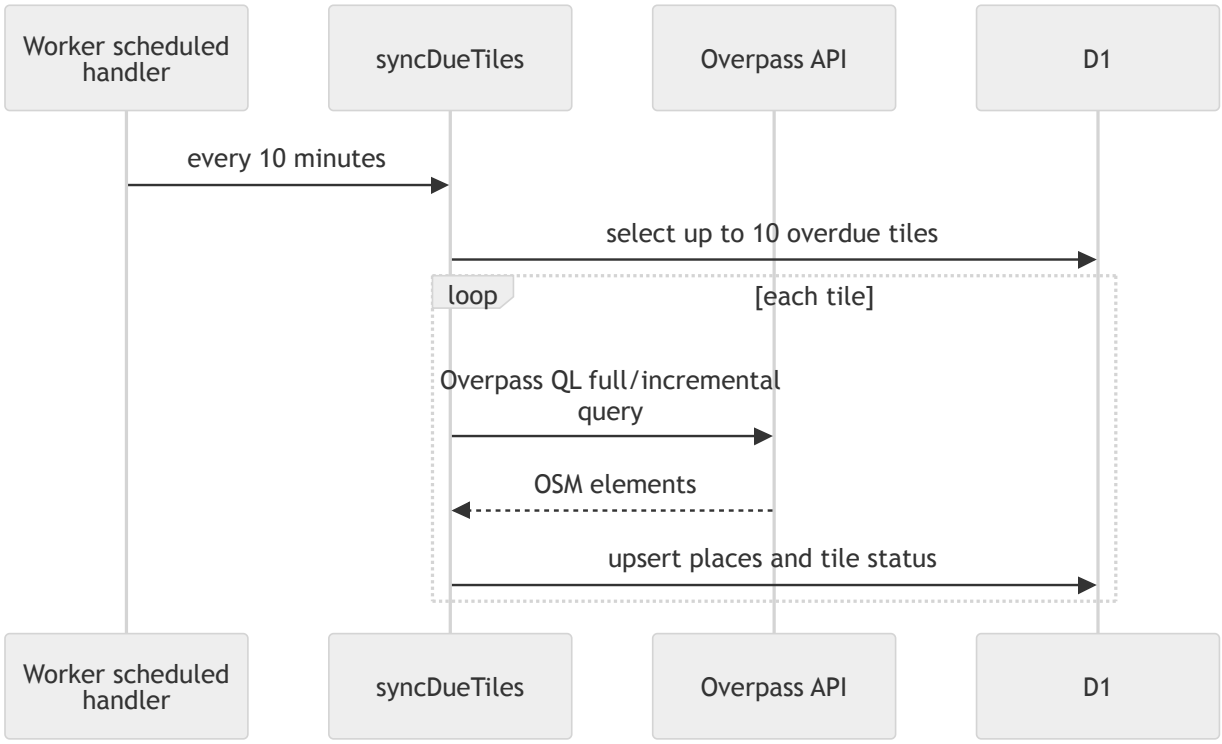


`UNIQUE(user_id, submission_id)` prevents duplicate reviews. Deterministic object keys make repeated uploads overwrite the same object. Writes are not automatically retried; the UI must reuse the same submission. D1 and R2 cannot form one transaction, so partial failures can leave orphaned objects.

Save/profile mutations

Saved-place toggles and profile changes update shared state optimistically and roll back on failure. The Worker authenticates with `requireUserId`, performs D1 writes, and returns the authoritative model.

Background place sync



Each populated tile is normally revisited after seven days; empty tiles after 28. Overpass busy responses stop the run and leave tiles due. There is no alternate provider or lock between manual and cron runs.

Failure scenarios

Scenario	Implemented response
No network / timeout	Shared GET retries twice; mutations expose error and require explicit retry
Auth expired	Auth/session state becomes signed out; protected requests return 401
Server unavailable	`ApiException`; map/detail surfaces retry, some profile/saved paths suppress errors
Malformed response	Kotlin serialization fails; `ignoreUnknownKeys` tolerates added fields
Third party down	Auth/email operation fails; Overpass delays directory freshness; Android map tiles may be blank
Duplicate write	Review ID constraint, deterministic media keys, and idempotent saved PUT/DELETE protect retries
Local/remote disagreement	No offline database; reload replaces in-memory state, optimistic writes roll back
Realtime/cross-device update	None; clients reload on screen lifecycle or explicit actions

SOURCE: DOCS/ARCHITECTURE/INTEGRATIONS.MD

Integrations

Integration matrix

Source	Destination	Protocol	Authentication	Data	Failure handling
iOS/Android shared client	Worker	HTTPS REST/JSON	Bearer for protected routes	places, profiles, reviews, saved places	GET retry; explicit mutation retry
Mobile media picker	Worker/R2	HTTPS PUT JPEG	Bearer	JPEG up to 10 MB	deterministic key; no automatic write retry
Worker	D1	Binding/SQL	Cloudflare binding	relational application/auth data	errors become HTTP failure; batches for related writes
Worker	R2	Binding	Cloudflare binding	JPEG objects	cross-store operation is not atomic
Worker	Overpass	HTTPS POST, Overpass QL	none	OSM place elements	busy errors stop run; ordinary tile errors retry later
Better Auth	Apple/Google	Provider protocol via Better Auth	configured client credentials	identity tokens/account data	provider errors flow to auth response
Better Auth	Resend	HTTPS JSON	`RESEND_API_KEY`	verification/reset email	request fails; local mode logs email
iOS	MapKit	Apple framework	OS/service-managed	tiles, map rendering, Look Around, directions	native framework behavior
Android	OpenFreeMap	HTTPS style/tiles through MapLibre	none	map style and tiles	no surfaced style-load recovery
Website	iOS	HTTPS Universal Link/AASA	signed app association	place-link query	website fallback
Website	Android	custom URI scheme	none	expected `place_id`	tracked page omits required ID (**potential issue**)

Data contracts

- Encoding: JSON over HTTPS except raw JPEG PUTs.
- Mobile uses Kotlin serialization with `ignoreUnknownKeys=true`, `explicitNulls=false`, and `coerceInputValues=true`.
- Backend request payload fields are camelCase; response DTOs map snake_case using `@SerialName`.
- Database timestamps are UTC ISO-8601 text (`strftime('%Y-%m-%dT%H:%M:%fZ','now')`); Better Auth uses integer timestamps in its own tables.
- Contribution writes explicitly send `version: 2`; backend preserves legacy projection fields.
- Most wire-domain codes are strings, so the backend is the final enum validator.

Verified contract findings

Finding	Status	Evidence
Shared API routes match the Worker dispatch paths	Verified	`RollspotApi.kt`, `backend/src/index.ts`

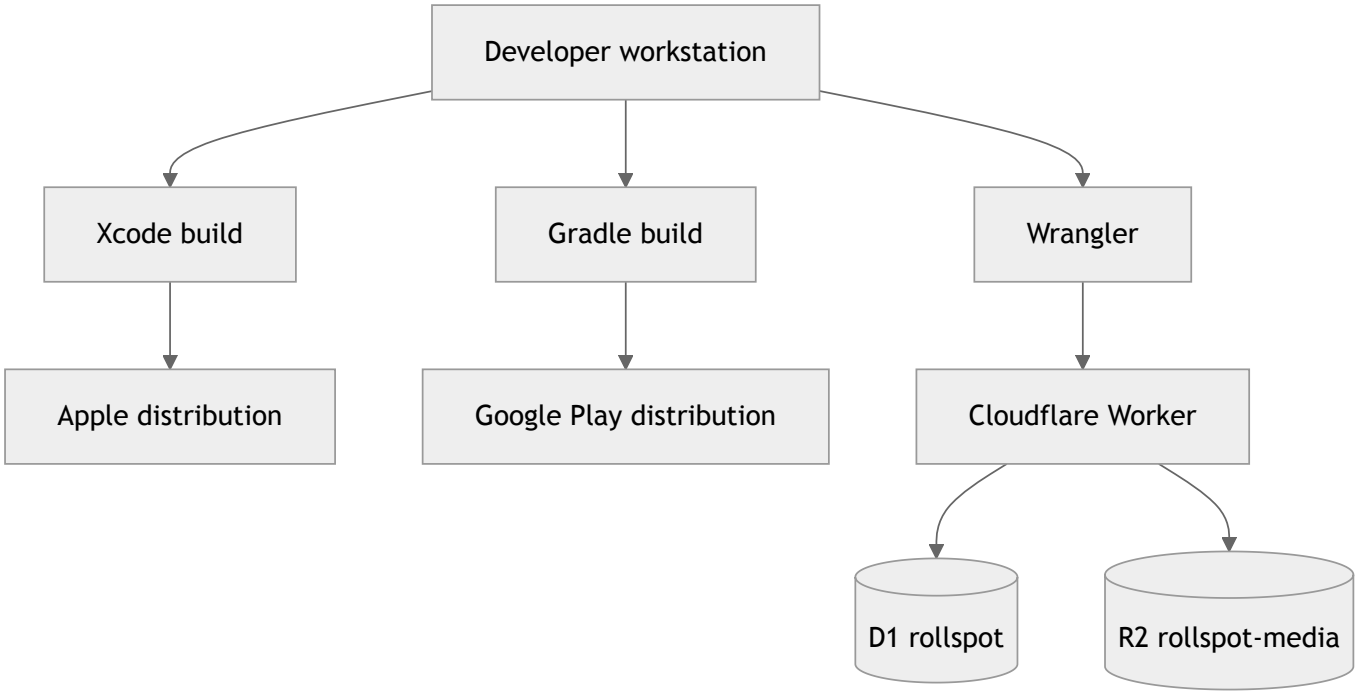
Finding	Status	Evidence
Shared client has no methods for health/admin/direct media reads	Verified/intentional	operational or URL-driven endpoints
Rich V2 review fields are returned but not modeled by `PlaceReview`	Potential issue	unknown keys are discarded, so clients use legacy projections
`show_real_name` is stored but review output does not consult it	Potential privacy issue	`profile.ts`, `reviews.ts`
Reviewer dedupe uses display name rather than user ID	Potential issue	`FacilityReview.dedupeKey`
API base URL differs between `backend/API.md` and deployed defaults	Potential issue	custom-domain status is unknown

Direct mobile integrations

iOS directly uses Apple MapKit, Core Location, Photos/UIImagePickerController, Keychain, Speech (currently unreachable), and Google Sign-In. Android directly uses MapLibre/OpenFreeMap, platform LocationManager, System Photo Picker, Android Keystore, and Credential Manager. Neither app talks directly to Overpass or a public OSM data API.

SOURCE: DOCS/ARCHITECTURE/DEPLOYMENT.MD

Deployment architecture



Verified runtime topology

- Worker name: `rollspot-api`; entry `backend/src/index.ts`.
- Worker compatibility date: `2026-09-01`; `nodejs_compat` enabled.
- D1 binding `DB` targets database name `rollspot`.
- R2 binding `MEDIA` targets bucket `rollspot-media`.
- Scheduled trigger: every ten minutes.
- iOS minimum is 18.0; Android min SDK is 26 and target SDK is 36.

Deployment interfaces

The repository supplies local commands, not an automated CI/CD pipeline:

```
cd backend
npm run db:migrate:remote
npm run deploy
```

Mobile distribution automation is not tracked. App Store Connect/Play Console configuration, signing certificate storage, Android upload-key custody, rollout policy, and rollback procedures are **Unknown**.

Configuration injection

- Worker plain variables and bindings are in `backend/wrangler.jsonc`.
- Worker secrets are set through Wrangler and represented by names in `.dev.vars.example`.
- Android local/release values are generated into `BuildConfig` from `Mobile/local.properties`.

- iOS reads Info.plist/build settings and accepts a debug `ROLLSPOT_API_BASE_URL` process override.
- Secrets must not be embedded in either app. Public OAuth client IDs are configuration, not secrets.

Migrations and rollback

D1 migrations are forward SQL files `0001` through `0005`. The deploy script does **not** run migrations; the operator must run the remote migration command separately. No down migrations, automated backup, restore verification, or rollback runbook is tracked. A Worker rollback cannot automatically reverse an already-applied schema change.

SOURCE: DOCS/IOS/README.MD

iOS application

Architecture

`RollspotApp` is the SwiftUI `@main` entry. It creates `LanguageManager` and `AuthSessionStore`, presents `SplashScreenView`, then `ContentView` / `HomeMapView`. `HomeMapView` owns the main `NavigationStack`; `HomeRoute` covers place, saved, contribute, and profile destinations. Auth uses a separate stack.

The app uses thin `ObservableObject` adapters over the static KMP `Shared` framework. Manual DI occurs in `Services/RollspotServices.swift`. SKIE exposes shared `StateFlow` as `AsyncSequence`; UI adapters are generally `@MainActor`.

Feature map

Feature	UI/entry	State/service	Server interaction / persistence
Home, map, search, filters	`Views/Home/HomeMapView.swift`, `SearchSheet.swift`	`MapStore`, shared `MapModel`	`/v1/places/nearby`, `/search`; MapKit renders
Place details	`Views/Detail/Mock/MockPlaceDetailView.swift`	`PlaceReviewStore`, `PlaceDetailModel`	place, photos, reviews endpoints
Contribution	`Views/Review/Mock/ContributeReviewFlowView.swift`	`ContributeFlowBridge`, `ReviewService`	photo PUTs then `POST /v1/reviews`; draft JSON in UserDefaults
Auth	`Views/Auth/`	`AuthSessionStore`, `AuthModel`	Better Auth; bearer in Keychain
Profile/account	`Views/Profile/`	`ProfileService`, shared profile models	profile, avatar, password, account/review deletion
Saved	`Views/Saved/SavedView.swift`	`SavedPlacesService`	`/v1/saved-places/*`
Photos	`Views/Photos/`	picker/camera and `ReviewService`	JPEG upload through shared review pipeline
Localization	all views	`LanguageManager`	in-code English/Indonesian dictionary, `@AppStorage`

Native capabilities

- MapKit annotations, camera, snapshots, Look Around, and Apple Maps directions.
- Core Location, when-in-use permission, one-shot location.
- Camera through `UIImagePickerController`; library through `PhotosPicker`.
- Keychain token (`AfterFirstUnlockThisDeviceOnly`).
- Associated domain `applinks:puspadi-fellas.pages.dev`.
- Sign in with Apple and Google Sign-In.
- Speech recognition/microphone implementation exists but no caller was found.

Networking and state

Most requests originate in shared `RollspotApi`; Ktor Darwin performs HTTP. GET network/timeout failures retry twice; writes require explicit retry. `ImageStore` uses `URLSession` and stores thumbnails under Caches plus decoded images in

`NSCache`. Bearer tokens are the only sensitive session state; draft review content and place metadata are unencrypted in UserDefaults.

`KeychainTokenStore.save` suppresses persistence errors with `try?`, so a successful server sign-in may not survive launch if Keychain writing fails (**potential issue**).

Deep links

`DeepLinkRouter` supports `puspadifellas://place`, the Pages `/place` URL, and `puspadi://auth/callback`. `RollspotApp.onOpenURL` first offers URLs to Google Sign-In, then place/auth routing. Query-supplied place metadata can be shown before canonical resolution (**potential issue**).

Build and configuration

- Deployment target: iOS 18.0; bundle `com.puspadifellas.app`; Swift setting 5.0.
- GoogleSignIn is resolved by Swift Package Manager (9.2.0 in `Package.resolved`).
- Xcode's first phase runs `Mobile/scripts/build-shared-for-xcode.sh`.
- Debug API override: `ROLLSPOT_API_BASE_URL`.
- `Mobile/Info.plist` still contains Google client-ID placeholders.

Known implementation gaps

- No iOS unit/UI test target and no privacy manifest.
- Raw MapKit POI selection can produce `preview:<UUID>` rather than a Rollspot place ID, causing backend detail/review requests to fail.
- Profile “ADD PHOTOS” has no upload handler.
- `ContributeView` is a TODO stub and appears unreachable; current contribution uses the mock-named live UI.
- `ReviewWizardView`, `ReviewViewModel`, `CloudflareAPIClient`, and `AccessibilityService` are legacy or unreferenced; `ReviewService` still holds an unused legacy client.
- `PlaceAISummaryService` intentionally returns no summary until a backend endpoint exists.
- Account-deletion feedback is collected but not sent.
- Several failures are printed without visible retry UI; no realtime refresh exists.

SOURCE: DOCS/ANDROID/README.MD

Android application

Architecture

`RollspotApplication.onCreate` constructs the app-wide `RollspotSdk` with `AndroidTokenStore` and restores the session. `MainActivity` handles intents and hosts the Compose `RollspotApp` `NavHost`.

Routes cover home, place detail, saved, auth, profile/settings/account/about, contribute search, and contribute flow. Thin ViewModels hold shared KMP models; Compose normally observes them with `collectAsStateWithLifecycle`.

Feature map

Feature	UI/ViewModel	Native dependency	Server interaction
Map/search/filter	`HomeScreen`, `HomeViewModel`, `PinMap`	MapLibre/OpenFreeMap, LocationManager	nearby/search
Place detail	`PlaceDetailScreen`, `PlaceDetailViewModel`	`geo:` directions, dial intent	detail/reviews/photos/save
Contribution	`ContributeFlowScreen`, `ContributeFlowViewModel`	System Photo Picker, JPEG encoder	media PUTs and review POST
Auth	`AuthFlowScreen`, `AuthViewModel`	Credential Manager/Google ID	Better Auth
Profile/account	`ProfileScreen`, `SettingsScreens`	Photo Picker	profile/avatar/reviews/account
Saved places	profile/saved route	none	saved-place endpoints

Native capabilities and storage

- MapLibre 13.6.1 loads the OpenFreeMap Liberty style.
- Fine/coarse location uses platform `LocationManager`, with an eight-second request timeout.
- System Photo Picker requires no storage permission; images resize to 2048 px at JPEG quality 80.
- Google identity uses Credential Manager.
- `AndroidTokenStore` encrypts the token with AES-GCM and a non-exportable Android Keystore key, storing ciphertext in private SharedPreferences.

No Room, DataStore, WorkManager, service, receiver, alarm, notification, camera capture, or background location is implemented. Contribution drafts and recent searches are in memory.

Configuration and build

- Application ID `com.rollspot.app`; min SDK 26; compile/target SDK 36; JVM 17.
- Gradle 9.1, AGP 8.13.2, Kotlin 2.4.20, Compose BOM 2026.05.01.
- `rollspot.apiBaseUrl` and `rollspot.googleWebClientId` come from `Mobile/local.properties`.
- Release cleartext traffic is blocked; debug permits localhost/`10.0.2.2`.
- Release minification is disabled.

`./gradlew :androidApp:assembleDebug :androidApp:testDebugUnitTest` passed during the audit, but there are no Android app test sources. Gradle reports deprecations relevant to Gradle 10.

Deep links

The manifest accepts `puspadi://auth` and `puspadifellas://place?place_id=...`. There is no verified HTTPS Android App Link or `assetlinks.json`. Place IDs containing `/` are interpolated directly into Navigation routes, which may break route matching. The tracked web place page omits the `place_id` expected by `MainActivity`, so its open-app button cannot resolve a place (**potential issue**).

Error behavior and gaps

- Map/detail provide retry; saved/profile initial loads sometimes suppress errors and display empty state.
- Map style-load failure has no user-facing recovery.
- Location permission is requested immediately and cached location has no age/accuracy gate.
- Contribution draft state is lost on process death despite UI language implying it is saved.
- UI says “Take or upload,” but only gallery picking exists.
- Contribution login uses an unmasked password field.
- Many strings are hard-coded English despite resource translations.
- `ReviewViewModel` is unused; active contribution calls the shared model directly.

Cross-platform parity

Capability	iOS	Android	Difference / gap
API place map/search/filter	Yes	Yes	MapKit vs MapLibre/OpenFreeMap
Place detail/reviews/photos/save	Yes	Yes	Native presentation differs
Six-category contribution	Yes	Yes	iOS persists a draft; Android draft is volatile
Camera capture	Yes	No	Android gallery only
Email auth	Yes	Yes	Shared AuthModel
Apple sign-in	Yes	No	Platform-specific
Google sign-in	Code present; config placeholder	Yes with config	Public client IDs required
Profile/avatar/review deletion	Yes	Yes	iOS profile photo add flow is incomplete
Shared HTTPS place links	Universal Links	No	Android custom scheme only
Offline database	No	No	Server state reload
Push/realtime	No	No	Explicit reload
Localization	In-code English/Indonesian	Partial resources; hard-coded English remains	Not full parity

SOURCE: DOCS/BACKEND/README.MD

Backend architecture

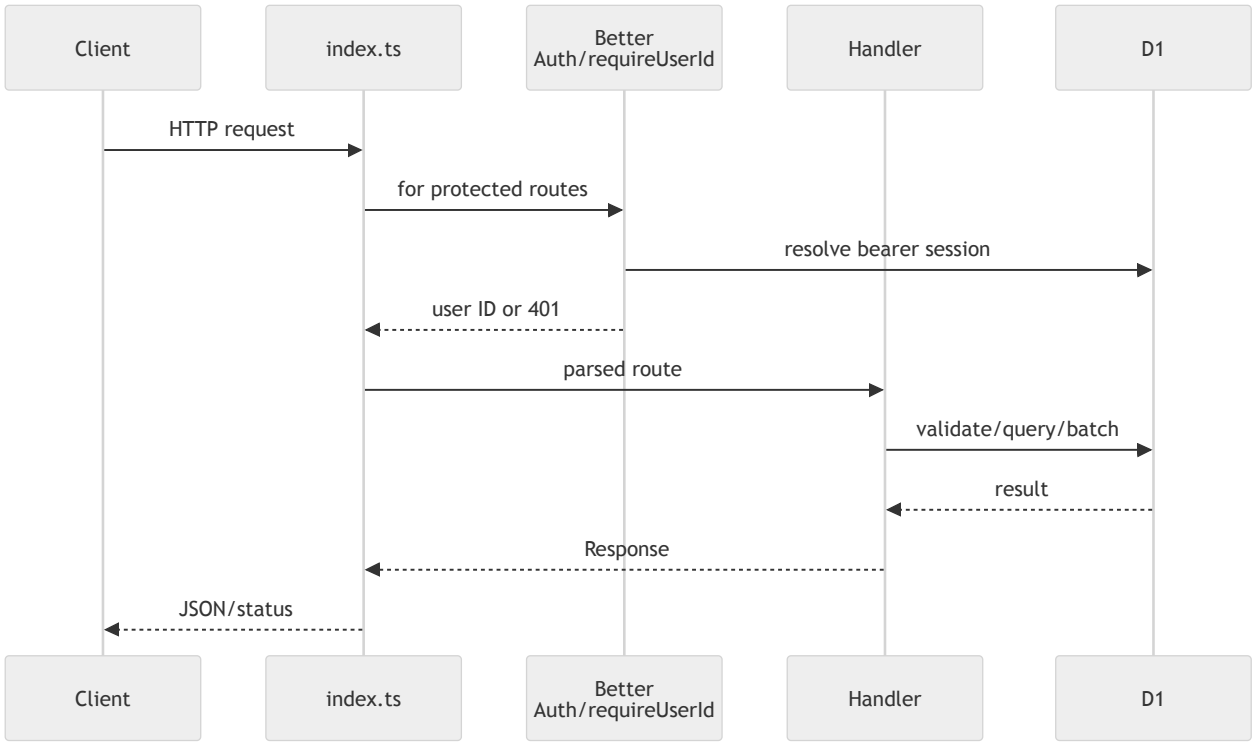
Runtime and lifecycle

The backend is a TypeScript Cloudflare Worker. `backend/src/index.ts` exports:

- `fetch(request, env)`: ordered path/method dispatch, error normalization, HTTP response.
- `scheduled(controller, env, ctx)`: `ctx.waitUntil(syncDueTiles(env))`.

There is no framework router or middleware chain. `http.ts` supplies `HttpError`, JSON responses, method checks, and JSON-body validation. JSON gets `Cache-Control: no-store`.

Request path



Expected failures use `HttpError`. Unexpected 5xx errors are logged and returned as JSON. The current handler exposes `error.message` for unexpected failures (**potential information-disclosure issue**).

Modules

Module	Responsibility
`auth.ts`	Better Auth, bearer sessions, Apple secret JWT, Resend mail
`places.ts`	place search/detail, grades/photos, saved places
`reviews.ts`	review create/read/delete, my reviews, public review output
`contribution.ts`	V2 payload validation and normalization
`media.ts`	deterministic JPEG upload and R2 reads

Module	Responsibility
`profile.ts` / `account.ts`	profile/avatar/password-facing state and account deletion
`sync.ts` / `osm.ts`	tiled scheduler and Overpass client
`env.ts`	bindings and variable/secret names

Authentication and authorization

Better Auth owns user/session/account/verification tables. Email/password requires verification; Apple and Google providers are enabled only when configured. Application handlers use `requireUserId`. Review delete checks ownership; V2 review photo references check media ownership. Admin sync compares an exact bearer `ADMIN_TOKEN`.

Public resources: place directory, reviews, review photos, and media-object reads. Protected resources: write/review operations, profile, saved places, provider list, and account deletion.

Place sync

The ten-minute cron processes at most ten overdue tiles. First load is full; later queries overlap the last load by 24 hours. Normal populated tiles are due after seven days, empty tiles after 28. Large responses/timeouts split tiles down to 0.01 degrees. Overpass 429/502/503/504 stops the run.

No lease prevents concurrent cron/manual work, no alternate Overpass endpoint exists, and no OSM deletion path removes vanished places.

R2 consistency

Uploads write R2 then D1 metadata. Deletes generally remove D1 state before deleting R2. D1 and R2 cannot commit atomically; orphaned objects are possible. No reconciliation or lifecycle job is implemented.

Configuration and observability

Plain variables/bindings are in `wrangler.jsonc`; secret names are in `.dev.vars.example`. Logging consists of unexpected request errors, tile failures, and a cron summary. No repository-configured traces, metrics, alerts, request IDs, crash tracking, or analytics exist. `/health` is liveness only.

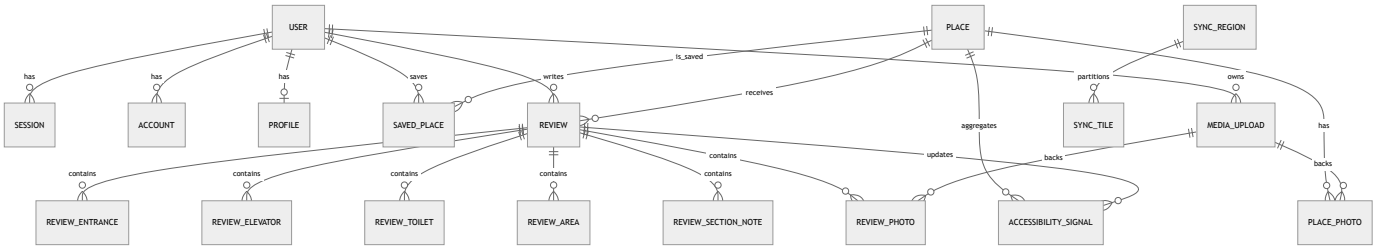
Tests

`npm test` and `npm run check` both run `tsc --noEmit`. No backend runtime test runner or test files exist. Behavioral coverage comes only from shared KMP tests and the environment-gated live backend journey.

SOURCE: DOCS/BACKEND/DATA-MODELS.MD

Data architecture

Relational model



Actual table names are lowercase/plural as defined in `backend/migrations`.

Domains and keys

Domain	Tables	Identity and constraints
Better Auth	`user`, `session`, `account`, `verification`	text IDs; unique email/session/provider identity
Directory	`places`, `places_fts`	integer FTS row ID, stable Rollspot UUID, unique `osm_ref`; lat/lng index
Sync	`sync_regions`, `sync_tiles`	region ID plus tile bounds/due state
Profile	`profiles`	one row/user; unique pseudonym; JSON-valid mobility aids
Saved	`saved_places`	composite `(user_id, place_id)`
Reviews	`reviews` and normalized child tables	unique `(user_id, submission_id)`; children cascade
Grades	`accessibility_signals`	one current `(place_id, user_id, feature)` signal
Media	`media_uploads`, `review_photos`, `place_photos`	R2 object key owns metadata; photo references restrict deletion

Foreign keys generally cascade user/place/review deletion. Photo rows use `ON DELETE RESTRICT` for media objects. Domain `CHECK`s protect feature/value codes and JSON validity.

Access patterns

- Nearby places: latitude/longitude bounds, then distance handling in `places.ts`.
- Search: FTS5 over name/address, maintained by insert/update/delete triggers.
- Place grade: group current accessibility signals by feature/value.
- Public reviews: place/time index; own reviews: user/time index.
- Saved places: user and reverse chronological creation.
- Sync: due-time and region indexes.

Consistency

Related D1 statements use `DB.batch`; no explicit SQL transactions appear. Review idempotency is enforced at the schema boundary. R2 plus D1 operations are not atomic. OSM sync upserts by `osm_ref`, retaining the Rollspot ID so reviews survive imports.

Data lifecycle

- Account deletion cascades relational user data, then attempts R2 deletion in batches.
- Review deletion cascades normalized rows and signals, then removes linked R2 objects.
- No retention duration, archival, backup/restore, soft delete, or R2 lifecycle policy is defined.
- OSM import has no deletion/tombstone pass.
- Public JSON is uncached; mobile state is memory-bound except token/draft/image caches.

Potential data issues

- `media_uploads.owner_user_id` has no dedicated index although account deletion filters on it.
- D1/R2 partial failure can leave orphaned media.
- `profiles.show_real_name` is not honored by public review output.
- Incremental OSM upsert cannot reflect deletions from the source.
- Production foreign-key enforcement and backup posture are **Unknown**.

SOURCE: DOCS/API/README.MD

API reference

Implementation authority: `backend/src/index.ts` and its handlers. Client authority: `Mobile/shared/.../api/RollspotApi.kt`. The manually maintained `backend/API.md` contains detailed payload examples but has known cron/base-URL drift. No OpenAPI specification is generated, and Better Auth owns a dynamic `/api/auth/*` surface.

Common error shape:

```
{"error":"error_code","message":"Human-readable message."}
```

Protected routes require `Authorization: Bearer <session-token>`. JSON responses use `no-store`.

Inventory

Method / route	Purpose	Auth / authorization	Inputs	Success	Known errors	Consumers	Source
`GET /health`	Liveness	Public	none	`{"status":"ok"}`	500	Operations	`index.ts`
`POST /v1/auth/email-registered`	Check account existence	Public	JSON `email`	`registered` boolean	400/415	Both apps	`index.ts`
`GET /v1/auth/providers`	Linked provider IDs	Bearer/self	header	provider list	401	Both apps	`index.ts`
`GET /v1/profile`	Current profile	Bearer/self	header	profile DTO	401	Both apps	`profile.ts`
`PATCH /v1/profile`	Update profile fields	Bearer/self	name/mobility/visibility	profile DTO	400/401	Both apps	`profile.ts`
`PUT /v1/profile/avatar`	Replace avatar	Bearer/self	JPEG, max 5 MB	profile/avatar response	400/401/413/415	Both apps	`profile.ts`
`GET /v1/saved-places`	List bookmarks	Bearer/self	none	places list	401	Both apps	`places.ts`
`PUT /v1/saved-places/:key`	Save place	Bearer/self	Rollspot/OSM key	saved state	400/401/404	Both apps	`places.ts`
`DELETE /v1/saved-places/:key`	Unsave place	Bearer/self	key	saved state	400/401	Both apps	`places.ts`

Method / route	Purpose	Auth / authorization	Inputs	Success	Known errors	Consumers	Source
`POST /v1/reviews`	Submit legacy or V2 review	Bearer/self	review JSON	existing/new review and grades	400/401/404	Both apps	`reviews.ts`, `contribution.ts`
`GET /v1/reviews/:id`	Read review	Public	review UUID	review DTO	400/404	Both apps	`reviews.ts`
`DELETE /v1/reviews/:id`	Delete own review	Bearer/owner	review UUID	status	401/403/404	Both apps	`reviews.ts`
`GET /v1/me/reviews`	Current user's reviews	Bearer/self	none	profile + review list	401	Both apps	`reviews.ts`
`DELETE /v1/me`	Delete account	Bearer/self	none	deleted status	401/500	Both apps	`account.ts`
`PUT /v1/media/review-photos/:place/:submission/:file`	Upload staged photo	Bearer/self	JPEG max 10 MB	object URL/key metadata	400/401/404/413/415	Both apps	`media.ts`
`GET, HEAD /v1/media/:objectKey`	Read media	Public	object key	JPEG/body metadata	404	URL consumers	`media.ts`
`POST /v1/admin/sync-places`	Run directory sync	Admin bearer	`tiles=1..200`, optional `region`	per-tile results	401/405	Operator	`sync.ts`
`GET /v1/places/nearby`	Nearby directory	Public	`lat`, `lng`, `radius`	places list	400	Both apps	`places.ts`
`GET /v1/places/search`	Search directory	Public	`q`, optional location/radius	places list	400	Both apps	`places.ts`
`GET /v1/places/:key`	Place detail	Public	Rollspot UUID or `osm:type/id`	place DTO	400/404	Both apps	`places.ts`
`GET /v1/places/:key/accessibility`	Grade detail	Public	key	feature grades	400/404	Legacy iOS only, currently unused	`places.ts`
`GET /v1/places/:key/photos`	Venue photos	Public	key	photo list	400/404	Both apps	`places.ts`

Method / route	Purpose	Auth / authorization	Inputs	Success	Known errors	Consumers	Source
`GET /v1/places/:key/reviews`	Place reviews	Public	key	review list	400; missing place may return empty	Both apps	`reviews.ts`
`GET /v1/places/:key/review-photos`	Flattened review photos	Public	key	photo list	400; missing place may return empty	Both apps	`reviews.ts`

Better Auth routes used by clients

Method / path	Purpose
`POST /api/auth/sign-up/email`	Create email account
`POST /api/auth/send-verification-email`	Send/resend verification
`POST /api/auth/sign-in/email`	Email/password sign-in
`POST /api/auth/sign-in/social`	Apple/Google identity token sign-in
`GET /api/auth/get-session`	Restore current session
`POST /api/auth/change-password`	Change password
`POST /api/auth/sign-out`	Revoke/sign out session

Exact Better Auth request/error schemas and additional vendor routes are **Unknown** without treating the installed package as a public Rollspot contract.

Key request contracts

Nearby requires numeric `lat` / `lng`; radius defaults to 5,000 and is clamped to 500–20,000. Search requires `q`. Place keys must be a Rollspot UUID or accepted OSM reference.

V2 contribution shape is defined by shared `ContributionSubmission` and parsed by `parseContributionV2`. It includes:

```
{
  "version": 2,
  "submissionId": "client-stable-uuid",
  "placeId": "rollspot-place-id",
  "persona": "wheelchair",
  "entrances": [],
  "elevators": [],
  "toilets": [],
  "areas": {},
  "sectionNotes": {},
  "finalText": null
}
```

Arrays/objects contain structured answers and staged media URLs. Allowed codes and size/count bounds are validated in `backend/src/contribution.ts`; clients should use `ContributeCodes` rather than free strings.

Compatibility

- API namespace is `/v1`; Better Auth uses `/api/auth`.
- Unknown response fields are ignored by clients.
- Required DTO fields and invalid types fail decoding.
- V2 responses currently include structured data the mobile `PlaceReview` DTO discards.
- Writes must retain stable `submissionId` and photo `fileId` across explicit retries.

Security notes

The public email-existence endpoint permits account enumeration. Media reads and reviews are public. There is no application rate limiter. See `../security.md`.

SOURCE: DOCS/SECURITY.MD

Security and privacy architecture

Implemented controls

Area	Verified implementation
Authentication	Better Auth email/password with verification, Apple/Google identity tokens, bearer sessions with signature requirement
Authorization	`requireUserId`; owner checks for review deletion/media references; separate `ADMIN_TOKEN`
iOS token	Keychain, `AfterFirstUnlockThisDeviceOnly`
Android token	AES-GCM key in Android Keystore; ciphertext in private SharedPreferences
Transport	Production URLs are HTTPS; Android release blocks cleartext; iOS ATS allows only local networking exception
Secrets	Backend secrets in Wrangler / ignored `.dev.vars`; secret names only in example
Data integrity	SQL foreign keys/checks/unique constraints; contribution validation; size/content-type bounds
Account deletion	`DELETE /v1/me` cascades D1 user data and attempts R2 cleanup
Media ownership	V2 submission verifies staged-photo ownership

Trust boundaries

Mobile apps are untrusted clients. The Worker validates auth, ownership, place existence, contribution codes, and media limits. D1 is not exposed directly. R2 is accessed only through the Worker, although media GET is public for anyone who knows an object key.

OAuth provider private credentials and Resend/admin secrets belong only in Worker configuration. Mobile apps contain public client identifiers.

Sensitive data

- Auth tables may contain password hashes/provider tokens managed by Better Auth; application code does not read them directly.
- Profiles can contain display name, mobility aids, avatar, pseudonym, and visibility intent.
- Reviews are public and link structured accessibility observations to a displayed reviewer identity.
- Session tokens are stored securely; iOS unfinished-review text is not encrypted.
- Worker logging can include local verification email HTML when Resend is absent; production logs should never use that fallback.

Potential issues

1. **Privacy:** `show_real_name` is persisted but public `reviewOutput` always returns `display_name` when present.
2. **Enumeration:** unauthenticated `/v1/auth/email-registered` reveals whether an email exists.
3. **Rate limiting:** none exists around auth, enumeration, uploads, or admin sync.
4. **Error exposure:** unexpected errors return their message to clients.
5. **Public media:** no signed URL or authorization; object-key obscurity is the only access boundary.
6. **Legacy review path:** legacy media URL ownership is not checked; V2 is protected.
7. **Token persistence:** iOS suppresses Keychain write failure.

8. **Draft privacy:** iOS stores review drafts unencrypted in UserDefaults.
9. **Privacy manifest:** no `PrivacyInfo.xcprivacy` was found.
10. **Backups:** Android allows backup; explicit data-extraction/backup policy was not found.

SOURCE: DOCS/TESTING.MD

Testing and reliability

Existing verification

Layer	Framework/task	Coverage observed	Audit result
Backend	`npm test`, `npm run check`	TypeScript type-check only	Passed by audit agent
Shared KMP	`commonTest` Kotlin tests	API retry/auth headers, auth/map/place/profile/saved/review logic, contribution rules	117 common + 3 gated live tests found; not rerun in one audit environment
Live backend	`LiveBackendTest`	full account/place/save/photo/review/idempotency/profile/delete journey	Runs only with environment variables
Android	`assembleDebug`, `testDebugUnitTest`	compilation; no Android app test sources	Passed during audit
iOS	Xcode app target	no unit/UI test target	Build not conclusively rerun during audit
Web	none	none	static inspection/live AASA check only

No CI workflow, coverage reporter, backend runtime test runner, screenshot tests, or end-to-end mobile UI suite is tracked.

Feature coverage map

Feature	Shared tests	Backend runtime tests	iOS tests	Android tests
Auth state/validation	Yes	No	No	No
API retry/error mapping	Yes	No	No	No
Place map/filter/detail	Yes	No	No	No
Accessibility grade rules	Yes	No	No	No
Contribution state/payload/idempotency	Yes	No	No	No
Saved/profile optimistic behavior	Yes	No	No	No
Secure storage	No	No	No	No
Deep links	No verified coverage	No	No	No
Cron/import	No	No	N/A	N/A
D1 migrations/constraints	Live journey partially	No dedicated test	N/A	N/A

Highest-value missing tests

- Backend handler and migration integration tests, including auth/ownership/error shapes.
- Public review identity test proving `show_real_name` is honored.
- V2 structured review read contract and per-pass photo attribution.
- R2/D1 partial-failure and orphan recovery.

- Token clearing on session 401/null and secure-store failures.
- Android route encoding for `osm:way/123`; web-to-app place link.
- iOS raw MapKit selection behavior and profile photo add path.
- Cron tile lease/concurrency and Overpass busy/oversize behavior.
- Both-app smoke UI tests for search, sign-in, contribution, and account deletion.

Reliability mechanisms

Implemented: GET network retry, stable write IDs, mutation gates, database uniqueness, optimistic rollback, bounded photo concurrency, scheduled retry through due tiles, and app state reload.

Absent: server retries/backoff for normal API requests, circuit breakers, queues, dead-letter handling, realtime sync, offline persistence, synthetic monitoring, alerting, and tested disaster recovery.

Commands

See `getting-started.md`. A feature is considered complete by repository convention only after backend type-check, shared tests, both app builds, and simulator/emulator acceptance checks.

SOURCE: DOCS/OPERATIONS/README.MD

Operations, deployment, monitoring, and troubleshooting

Release sequence

1. Review and apply new D1 migrations with `npm run db:migrate:remote`.
2. Type-check the Worker with `npm run check`.
3. Deploy with `npm run deploy`.
4. Run shared tests and build both mobile apps.
5. Smoke-test health, place search/detail, auth, contribution/photo upload, save, and account deletion.
6. Publish mobile builds through the relevant store process.

The repository has no CI/CD, staged rollout, or automated rollback. Mobile signing/distribution procedure is **Unknown**.

Configuration

Name	Purpose	Storage
`API_BASE_URL`	public canonical Worker URL	Wrangler variable
`BETTER_AUTH_SECRET`	Better Auth signing	Wrangler secret
Apple IDs/team/key/private key	Apple identity provider	variable + secrets
Google iOS/Android/Web IDs and secret	Google identity	public IDs + backend secret
`RESEND_API_KEY`, `EMAIL_FROM`	transactional email	secret + variable
`ADMIN_TOKEN`	manual sync authorization	secret
`DB`, `MEDIA`	D1/R2 bindings	Wrangler config

Observability

Verified signals:

- `GET /health` returns process liveness.
- Worker logs unexpected 5xx, sync tile errors, and a scheduled-run summary.
- D1 `sync_regions` / `sync_tiles` contains freshness/status fields.

Not implemented in repository: traces, metrics, dashboards, alert rules, request IDs, external uptime checks, crash reporting, analytics, or structured request logs.

Runbooks

Place search is empty or stale

1. Call `/health`.
2. Check Worker scheduled logs and D1 due/status fields.
3. Verify Overpass is responding; 429/502/503/504 intentionally stop a run.

4. Run `POST /v1/admin/sync-places?region=bali&tiles=<n>` with the admin bearer token.
5. If only Android's map background is blank, test OpenFreeMap separately; directory/list data may still work.

Sign-in fails

1. Test `/v1/auth/email-registered` and Better Auth session endpoint.
2. Determine whether all methods or one provider fails.
3. Verify Apple/Google client IDs, Apple key material, callback origins, and Resend sender status.
4. For iOS Google failure, confirm the Info.plist placeholders were replaced.
5. Do not log identity tokens or bearer tokens.

Review/photo submission fails

1. Preserve and reuse the same `submissionId` and `fileId`.
2. Check the first failed media PUT before the review POST.
3. Inspect Worker validation error code/message and verify the place key exists.
4. Retry explicitly; writes are not automatically retried.
5. If D1 failed after R2 upload, record potential orphan cleanup; no automated reconciler exists.

Shared links fail

1. iOS: fetch the AASA, verify JSON content type/app ID, associated-domain entitlement, and `/place*`.
2. Android: custom scheme requires `place_id`; HTTPS App Links are not implemented.
3. Compare deployed Pages content with tracked `web/`; deployment drift was observed.

Deployment or migration failure

1. Stop before deploying incompatible code.
2. Inspect Wrangler migration output and remote migration list.
3. There are no down migrations: restore/forward-fix decisions require an operator and backup knowledge.
4. Worker rollback does not revert D1.

Account deletion partially fails

D1 deletion can succeed before R2 cleanup fails. Re-authentication may no longer be possible although orphaned objects remain. Confirm the account is deleted, collect affected object keys from logs if available, and remove only verified orphaned objects.

Recovery limitations

Backup locations, RPO/RTO, D1 restore rehearsal, R2 versioning/lifecycle, store rollback, and incident escalation contacts are not defined in the repository.

Architecture decisions and extension guide

These records describe decisions evidenced by implementation. Historical rationale is marked **Unknown** when it cannot be recovered.

One backend and shared KMP business logic

- **Context:** two native apps require consistent API contracts, validation, auth state, and grade rules.
- **Decision:** centralize these in `Mobile/shared`; keep SwiftUI/Compose and platform SDK work native.
- **Trade-off:** consistent logic and tests, at the cost of Gradle/Xcode integration and Swift bridging.
- **Evidence:** `CLAUDE.md`, `RollspotSdk`, shared feature models, native adapters.

Own place directory sourced from OpenStreetMap

- **Decision:** apps query D1 via `/v1/places/*`; a Worker cron imports OSM through Overpass.
- **Consequences:** stable Rollspot IDs and cross-platform parity; attribution required; directory freshness depends on scheduled import and Overpass.
- **Rationale:** cross-platform source-of-truth need is evidenced; original product/business rationale is **Unknown**.

Native map renderers

- **Decision:** MapKit on iOS, MapLibre/OpenFreeMap on Android; neither chooses which places exist.
- **Trade-off:** platform-native experience and no Google Maps dependency, with visual/availability differences between platforms.

D1 plus R2

- **Context:** the Worker must store structured application data (users, places, reviews, grades) and binary JPEG media. A single storage type is a poor fit for both.
- **Decision:** Cloudflare D1 holds the relational database; Cloudflare R2 holds photo bytes. The Worker is the only component that talks to either store.
- **Why D1:** D1 is SQLite at the edge. Rollspot needs SQL joins, uniqueness constraints (for example one review per user and `submission_id`), indexes for nearby search and FTS, and foreign-key cascades when a user or review is deleted. Those are database concerns, not object-storage concerns. Schema and constraints live in `backend/migrations/`; query and batch usage live in Worker modules such as `places.ts`, `reviews.ts`, `auth.ts`, and `sync.ts`. The binding name is `DB` in `backend/wrangler.jsonc`.
- **Why R2:** review photos and avatars are opaque JPEG blobs, often several megabytes. Storing them as database BLOBs would bloat D1 and complicate streaming. R2 stores objects by key; D1 only stores the `object_key` and ownership metadata in `media_uploads`. Upload and read handlers are in `backend/src/media.ts` and avatar handling in `backend/src/profile.ts`. The binding name is `MEDIA` and the bucket is `rollspot-media` in `backend/wrangler.jsonc`.
- **How they connect:** a contribution uploads JPEG bytes to R2 first, records the key in D1, then the review POST references those keys. Public photo URLs point at `/v1/media/:objectKey`, which the Worker serves from R2. Account and review deletion remove D1 rows and then attempt matching R2 deletes.
- **Trade-off:** efficient domain queries and object storage, but D1 and R2 cannot commit in one transaction, so a partial failure can leave an orphaned object or a deleted row with leftover bytes.
- **Where to look next:** full table map in `../backend/data-models.md`; media flow in `../architecture/data-flow.md`.

Idempotent client-generated review IDs

- **Decision:** client keeps `submissionId` and photo `fileId`; D1 uniqueness and deterministic R2 keys absorb retries.
- **Consequence:** writes need no automatic retry, but the UI must preserve IDs after failure.

Technical debt

High

Finding	Evidence	Impact
Public reviews ignore `show_real_name`	`profiles` stores it; `reviewOutput` does not read it	User privacy preference may not be honored
Android place IDs are not route-encoded	Navigation interpolates IDs containing `/`	OSM-key detail/contribute routes may fail
Tracked Android open-app link omits required `place_id`	`web/place/index.html` vs `MainActivity.handleDeepLink`	Button cannot resolve place
Raw MapKit POI can produce `preview:<UUID>`	iOS selection/model code vs server place-key requirement	Detail/review API requests likely 404

Medium

Finding	Evidence / consequence
Backend has no runtime tests	`npm test` is TypeScript compilation only
No CI/CD or production observability	no workflows; logs/health only
V2 review read model is discarded	client `PlaceReview` omits structured fields and ignores unknowns
D1/R2 operations are non-atomic	orphaned media possible on partial failure
Account enumeration	public email-registration check reveals existence
No application rate limiting	auth/upload/admin surfaces depend on external controls, if any
Android draft is volatile	process death loses answers/photos/submission ID
iOS Google config is placeholder	provider fails unless release process substitutes values
iOS profile add-photo flow incomplete	state exists without handler/upload
Web deployment drift	deployed Pages content differs from tracked web files
OSM removals never delete places	incremental/full sync only upserts
Cron/manual sync can race	no tile lease/lock

Low / maintainability

- Legacy/unreferenced iOS clients, screens, and services remain.
- Live production Swift types retain `Mock` names.
- Android strings are only partially localized.
- Android release minification is disabled; Gradle 10 deprecations are reported.
- Xcode shared-framework phase has no declared inputs/outputs and runs every build.
- Reviewer dedupe uses display name rather than stable user identity.
- Several profile/saved failures become empty states instead of visible errors.
- Unexpected backend errors expose their message.

- Media upload buffers entire request before size validation.
- API documentation and cron comments have drifted from configuration.

Recommended order

1. Correct reviewer identity privacy and broken place-link/ID flows.
2. Add contract/backend tests around auth, contribution, review output, and migrations.
3. Add request correlation, structured logging, health/sync alerting, and rate limits.
4. Reconcile V2 read DTOs and remove verified legacy code.
5. Define R2 orphan reconciliation and D1 recovery procedures.
6. Establish CI, release provenance, and documented mobile signing custody.